

BKCLTrade

How the codebase actually works

A semi-autonomous algorithmic trading laboratory for Indian markets (NSE / BSE) via Zerodha Kite Connect.

Generated 2026-06-03 • ~25,000 LOC • 720 passing tests
Two-version walkthrough: trader's view + plain-English view

Table of contents

§	Section	Page
	Cover	1
	Table of contents	2
A	Version A — for someone with trading knowledge	3
A.1	Core philosophy	3
A.2	The three concentric rings	3
A.3	The daemon — what runs when	5
A.4	The Indian-specific pieces	6
A.5	Hard safety rules	7
A.6	How an order actually flows	7
B	Version B — for someone without trading knowledge	8
B.1	What this project IS	8
B.2	The hospital metaphor	8
B.3	The three big chunks	8
B.4	The robot's daily schedule	10
B.5	The Indian-specific parts	11
B.6	The five rules that cannot be broken	11
B.7	What's happening right now	12
B.8	What would it take to make a real trade	12
C	Appendix	13
C.1	Repository map	13
C.2	Recent commits (India migration)	13

Version A — for someone with trading knowledge

Concrete jargon allowed. Assumes familiarity with STT, SPAN, T+1 settlement, walk-forward backtests, deflated Sharpe ratio, probability of backtest overfitting (PBO), BH-FDR, MWPL, and the broad idea of a hash-chained event ledger.

A.1 Core philosophy

*Build an autonomous systematic trading laboratory, not a chatbot that trades. AI generates hypotheses, mutates strategies, runs research, writes postmortems, and recommends promotions. A **deterministic** risk, ledger, validation, and execution kernel decides whether anything may trade.*

Autonomy level: L2 — the bot may run sandbox experiments and produce validation packets; it cannot promote to live capital without human sign-off.

A.2 The three concentric rings

Ring 1 — Deterministic kernel (no LLM ever)

Module	LOC	What it does
ledger/	3,216	27 SQLite tables, append-only + hash-chained. Each row carries prev_hash and this_hash = sha256(prev_hash ■ canonical_json(row)). Same primitive as a blockchain. Verified at boot and nightly.
risk/	2,912	Single-entry gate precheck.evaluate(). 8 kill switches × 7 cap categories × 4 SEBI gates × regime overlay × live-capital cap. Strictest rule wins.
execution/order_router	—	Risk-gated, idempotent (client_order_id-keyed), data-freshness-checked submission with exponential-backoff retry on transient broker errors. Permanent errors fail fast.
kernel/boot.py	177	At startup: SQLite integrity_check on ledger + mirror; schema version match; recompute SHA-256 of every .lock against policy/HASHES; verify the full hash chain; surface active kills. Refuses to start on any mismatch.

Off-host mirror DB for cross-checking + single-writer file lock to prevent racing writers.

Ring 2 — Strategy registry + validated alpha

registry/ ships three append-only tables: **strategy_version** (immutable code+config hash per version), **validation_artifact** (three tiers per Plan §13), **promotion_packet** (Tier-3 with human sign-off).

Nine strategies are live in code:

- **Cross-sectional momentum** — etf_momentum_v1/v3 (12-1 month rank on NSE ETFs, top-N equal weight, monthly rebalance)
- **Dual momentum** — dual_momentum_v1/v3 (binary rotation NIFTYBEES ↔ LIQUIDBEES based on absolute momentum)
- **Crypto trend** — crypto_momentum_v1/v3 (INR pairs via CoinDCX, capped at 15% equity)

- **Options wheel** — `spy_wheel_v1/v3` (CSP → assigned → CC, multi-underlying NIFTY/BANKNIFTY/FINNIFTY in v3)

- **NIFTY gap fade** — `nifty_gap_v1` (intraday hypothesis at `research_only`; just added)

A lane only emits orders when `status ∈ {tiny_paper, scaled_paper, live}`. `research_only`, `shadow`, `halted`, `observe_only`, `reduce_only` are blocked from new entries.

Ring 3 — Research factory (LLM-allowed, sandboxed)

`research/` (~4,900 LOC, 28 modules) is a full quant research stack:

- **walkforward.py** — purged + embargoed walk-forward folds + 30% locked holdout
- **dsr.py** — Deflated Sharpe Ratio (Bailey / López de Prado), `n_trials-aware`
- **pbo.py** — Probability of Backtest Overfitting (same authors' combinatorial test)
- **bh_fdr.py** — Benjamini-Hochberg multiple-testing correction; writes adjusted p-values + survived flag
- **parameter_plateau.py** — best Sharpe shouldn't be a needle in noise
- **failure_memory.py** — 90-day reject cache, hash-chained, append-only; mutation engine cannot re-propose recent duds
- **mutation_engine.py** — drives nightly cycles from `policy/search_space_v1.json` (hash-locked); 64 variants/family/month cap
- **sandbox.py** — blocks `execution/`, `kernel/`, and `risk.precheck` imports during mutation runs (LLM physically cannot touch the gate)
- **paper_validation.py** — `precheck` → `submit` → `fill` → `slippage compare` for fast-tracked candidates
- **tier1.py** — `backtest` → `DSR + PBO` → `validation_artifact`

Promotion happens **only** when Tier-3 packets pass; the kernel rejects any version with `validation_artifact_id = NULL` or a failing artifact.

A.3 The daemon — what runs when

`daemon/scheduler.py` wires 17 jobs on APScheduler. Cadences from Plan §6 + §10:

Job	Cadence	Purpose
<code>boot_check</code>	startup + every 6h	Re-verify policy hashes + hash chain
<code>market_data_ingest</code>	every 1 min during RTH, 5 min otherwise	Pull bars, update lane watermarks
<code>position_snapshot</code>	every 5 min	Capture current broker positions
<code>account_snapshot</code>	every 5 min	Equity / cash / buying-power; session-start anchor
<code>orphan_loop</code>	every 30 s	Reconcile broker orders we didn't initiate
<code>reconciliation</code>	23:00 IST nightly	Ledger vs broker cross-check; fire <code>recon_mismatch</code> kill on diff
<code>drift_monitor</code>	23:30 IST nightly	Modelled vs realised cost over last 20 fills; demote on tolerance
<code>strategy_runner</code>	15:30 IST daily, 7d/wk	Fire each strategy's <code>evaluate_strategy()</code> ; equity self-skips on holidays
<code>mutation_cycle</code>	nightly across v3 families	Propose → backtest → BH-FDR
<code>universe_audit</code>	daily	Detect drift between live universe and policy-locked filter
<code>regime_monitor</code>	daily	Classify regime; write <code>regime_event</code>
<code>intel_refresh</code>	daily	Pull RBI / VIX / G-Sec / F&O / BSE feeds
<code>mutation_review</code> / <code>source_scout</code> / <code>strategy_intake</code> / <code>search_space_proposal</code>	various	Phase D research-bot scaffold

A.4 The Indian-specific pieces

We translated the entire system from a US-market design (NYSE / Alpaca / SEC / FINRA) to an India-market design (NSE / BSE / Zerodha / SEBI / STT / GST). What landed:

Concern	Implementation
Broker	Zerodha Kite Connect adapter at <code>ingest/zerodha_adapter.py</code> (15.5 KB scaffold). Access token refreshes daily at 06:00 IST.
Cost model (3 lenses)	Raw / broker_paper / pessimistic. Gate uses pessimistic only. Stocks (CNC): STT 0.1% both sides + stamp 0.015% buy + exchange 0.00325% + SEBI 0.0001% + GST 18% on (brokerage+exchange+SEBI). F&O: $\blacksquare$ 20 flat + STT 0.0125% sell + stamp 0.003% buy + GST. Crypto: 20 bps taker + 1% TDS on sells $\geq$ $\blacksquare$ 10k (Section 194S).
Calendar	NSE holidays hand-curated through 2027 (Republic Day, Mahashivratri, Holi, Eid, Good Friday, Ambedkar, Maharashtra, Independence, Ganesh Chaturthi, Gandhi Jayanti, Dussehra, Diwali, Gurunanak, Christmas). Muhurat session treated as closed for cadence. No half-days.
Session clock	RTH 09:15-15:30 IST. Pre-open 09:00-09:15 (call auction; orders accepted, no execution). IST is fixed UTC+5:30, no DST.
SEBI circuit breakers	<code>risk/india_sebi.py</code> : tiered NIFTY halt (10/15/20%, time-of-day dependent), F&O ban-list opening-block (closing allowed), per-stock upper/lower price band check, conservative F&O buy-side margin guard (NOT SPAN — Zerodha computes real SPAN at gateway).
Crypto tax	30% flat on gains (Section 115BBH) tracked annually; 1% TDS on sells per-trade in cost model.
Intel feeds	All free. RBI rates (DBIE), India VIX (yfinance), G-Sec curve (FBIL), NSE F&O OI/PCR, BSE filings.

A.5 Hard safety rules

Cannot be bypassed without a deliberate env flag *and*, for some, a signed policy lock + 7-day cooldown:

- **No LLM in trading hot path** — `TRADING_BOT_ENABLE_LLM_HOTPATH` default off. Every entry / scout / unblock early-exits.
- **No live param writes** — `TRADING_BOT_ALLOW_LIVE_PARAM_WRITES=1` required.
- **Crypto $\leq$ 15% equity** — enforced by `risk.asset_class_caps`.
- **Append-only** — schema forbids UPDATE/DELETE on `order_master`, `fill_event`, `position_snapshot`, `strategy_decision`, `reconciliation_proof`.
- **Policy hash verification** at every boot. Loosening any threshold = new dated .lock file + signature + 7-day cooldown.

A.6 How an order actually flows

```
Strategy (e.g. etf_momentum_v1.evaluate_strategy) ↓ produces OrderIntent {symbol, qty, side, asset_class, ...}
execution.order_router.submit_order ■■ 1. risk.pcheck.evaluate ■ (kill switches → SEBI gates → account caps → PDT → ■ asset-class caps → lane caps → strategy loss → ■ regime overlay → symbol cap → order cap → live-cap) ■ → halt | reduce | accept ■■ 2. ingest.watermarks.check_lane_freshness → halt if stale ■■ 3. ledger.order_master.check_idempotent → reject dup ■■ 4. ledger.insert_order_master + state_event → hash-chained ■■ 5. broker_submit(client_order_id, ...) ■ with exp-backoff retry on transient errors ■■ 6. ledger.append_state_event(submitted | rejected) + mirror
```

Every step writes to the hash-chained ledger. Three days from now the entire decision can be replayed deterministically from the SQLite files alone.

Version B — for someone without trading knowledge

Plain language. Analogies. No jargon without immediate definition. Some sentences will sound oversimplified to a trader — that's deliberate.

B.1 What this project IS, in one paragraph

This is a robot that's *learning* how to trade Indian stocks and crypto. But it's a paranoid robot. It has a research lab where it can try out trading ideas on fake money, and a separate real money door that's bolted shut with five locks. The robot is allowed to run experiments freely. It is NOT allowed to spend real money without a human unlocking the door. **It is currently not spending any real money.** Everything you see is either a simulation or a paper-trading dry run.

B.2 The big metaphor — a hospital, not a casino

Hospital	This codebase
The pharmacy where drugs are stored	The ledger — every event recorded, nothing ever deleted
The triage nurse at the ER door	The risk kernel — every order is checked before it goes anywhere
The doctors writing prescriptions	The strategies — code that decides buy this, sell that
The research wing running clinical trials	The research factory — backtests, statistical validation
The hospital's medical board approving new procedures	The strategy registry — nothing goes live without a passed validation packet
The night janitor checking every door is locked	The boot check — at every startup, verify nothing has been tampered with

The whole thing is designed so that **the AI parts can fail or even go rogue and the system still cannot lose your money.**

B.3 The three big chunks

Chunk 1 — The vault (the deterministic kernel)

Three things live here, and they are written by humans only. The AI is forbidden from touching them.

The ledger is like a bank's transaction book, except it's a special book where you cannot erase pages. Every page has a fingerprint that depends on the previous page's fingerprint — if anyone tries to change page 47, the fingerprint on page 48 won't match anymore. The system catches this every time it starts up. This is the same idea Bitcoin uses, just simpler.

The risk kernel is a checklist. Before any order can be sent to your broker, it has to pass through this checklist:

- Are there any kill switches tripped right now? (data missing, broker down, etc.)
- Is the Indian stock market crashing? (the SEBI circuit-breaker rules — 10% / 15% / 20% halt tiers)
- Is this stock on today's do not trade list? (NSE publishes this daily for over-positioned scrips)
- Is the price allowed to move that high or low today? (every Indian stock has daily price bands)
- Are we putting too much into one stock?

- Are we putting too much into crypto? (cap: 15% of total money)
- And several more, including a per-order risk-of-loss cap.

If even **one** check fails, the order is blocked. No exceptions.

The execution layer is the actual messenger that takes an order to the broker (Zerodha). It tags every order with a unique ID so if the network hiccups and the order accidentally gets sent twice, the broker recognises the duplicate and ignores it. If the broker says I'm busy, try again, it retries 3 times with increasing delays. If the broker says this order is invalid, it gives up immediately.

Chunk 2 — The doctors (the strategies)

These are the actual trading ideas. There are 9 of them right now. Examples:

- **ETF momentum** — Buy whichever Indian stock fund has been going up the most for the past year. Once a month, check again and swap if a different one is winning. (Like always sitting at whichever cafeteria table is most popular.)
- **Dual momentum** — Either invest in NIFTYBEES (an Indian stock fund) OR in LIQUIDBEES (basically cash), based on which one is rising faster. Swap as needed. (Park-or-drive strategy.)
- **Crypto trend** — Buy Bitcoin or Ethereum (in rupees) when they're trending up. Never put more than 15% of your money here, because crypto is wild.
- **Options wheel** — A more advanced strategy that sells insurance contracts on NIFTY and collects premiums.
- **NIFTY gap fade** (just added) — When the Indian market opens at 9:15 AM unusually far from where it closed yesterday, bet that it will drift back toward yesterday's price by 3:30 PM.

All 9 strategies are currently in research only mode. None can buy or sell anything until a human approves them.

Chunk 3 — The lab (the research factory)

This is where the AI is allowed to play. Its job is to:

- Take a trading idea
- Run it against years of historical price data (backtesting)
- Statistically prove the idea isn't just lucky noise (using techniques borrowed from real quant finance research papers — the deflated Sharpe ratio, the probability-of-backtest-overfitting test, etc.)
- Try thousands of small variations of the idea (mutations) to find which works best
- Remember every variation that failed for 90 days so it doesn't propose the same dud twice
- Write up a validation packet — basically a report card

A human reads the report card and decides whether to promote the idea from lab experiment to shadow trading (still no real money but watched closely) to tiny paper (very small fake-money trades for 60+ days) to live (real money — currently blocked entirely).

The lab is in a sandbox. It literally cannot import the code that talks to the broker. If the AI somehow decides let me just send a buy order from here, Python refuses to even load the module.

B.4 The robot's daily schedule

The system runs as a long-living background process (a daemon). Different jobs fire at different times:

When	What
Every 30 seconds	Check for any orphan orders sitting at the broker that we don't know about (e.g. from a previous crash); reconcile them.
Every 1 minute (Indian market hours)	Pull fresh price data.
Every 5 minutes	Snapshot what we currently own.
Every 6 hours	Re-run the startup integrity checks. If anything has been tampered with, halt.
3:30 PM IST daily	Let each strategy decide what it wants to do tomorrow.
11:00 PM IST daily	Compare what we thought would happen today with what actually happened. If the gap is too big, demote the strategy.
11:30 PM IST daily	Compare costs (broker fees, slippage) we modelled vs what we actually paid. If we under-estimated costs, demote.
First of every month	Run a fresh round of mutation experiments in the lab.

B.5 The Indian-specific parts

Indian markets work differently from American markets in many small ways. We translated the whole system from a US-market design to an India-market design:

What's different in India	What we changed
The broker (Zerodha, not Robinhood / Alpaca)	New adapter to talk to Zerodha's API
Market hours (9:15 AM – 3:30 PM IST, no half-days)	New calendar + clock module
Costs (STT, GST, stamp duty, exchange fees, SEBI fees — not US-style SEC / FINRA fees)	Rewrote the entire cost model. Previously it was silently computing zero fees because the code still used American fee names — strategies looked artificially profitable.
Tax (30% flat on crypto gains, 1% TDS deducted at source on every crypto sale ≥ ₹10,000)	Built into the crypto cost model
SEBI's circuit-breaker rules (market halts at 10%, 15%, 20% drops depending on time of day)	New safety check
Daily F&O ban list (NSE bans new positions in over-positioned scrips)	New safety check
Per-stock daily price bands (every Indian stock has a max % move allowed per day)	New safety check
Free data sources only (RBI website, India VIX, NSE data, BSE filings)	5 new intel feed modules

B.6 The five rules that cannot be broken

These are the system's vows. The code is structured so that violating them requires deliberately editing source code AND flipping environment variables AND, for some, waiting 7 days:

- **The AI cannot directly place trades.** Ever. The trading path checks an env variable before allowing AI input; default is off.

- **The AI cannot change the trading rules at runtime.** Same env-var gate.
- **Crypto can never be more than 15% of the total money.** Enforced before every order.
- **Nothing in the ledger can be deleted or modified.** The database schema literally forbids it.
- **The rules can only be loosened with a 7-day cooldown and a signed lock file.** The startup check verifies the rules haven't been tampered with.

B.7 What's actually happening right now

- Zero real money is being traded.
- The system is set up for paper trading (fake-money simulation), but not even that is running because there's no `.env` file with broker credentials.
- 9 strategies are coded and registered, all in research only lane.
- The research lab is functional but has no historical price data to chew on yet — someone needs to write a job that downloads NSE price history.
- 720 automated tests pass, meaning the parts that ARE built work correctly.
- The whole architecture is solid. What's missing is operational wiring (data, credentials, monitoring).

B.8 What would have to happen for it to make a single real trade

In order:

#	Step
1	Download NSE historical data into <code>data/historical_bars.db</code> .
2	Run one of the strategies through the full Tier-1 → Tier-2 → Tier-3 validation pipeline (months of paper-trading observation required).
3	Get a human to sign the Tier-3 promotion packet.
4	Generate Zerodha API credentials and place them in <code>.env</code> .
5	Flip the lane status from <code>research_only</code> to <code>live</code> in the registry.
6	Set <code>BOT_MODE=live</code> and <code>ENABLE_SUBMIT=true</code> .
7	Restart the daemon. Boot checks run; if anything is amiss, refuses to start.
8	The first time a strategy fires after that, an order goes to Zerodha.

That's 7+ deliberate steps. The whole point of the architecture is that you cannot get to step 8 by accident.

Appendix

C.1 Repository map (LOC per package)

Package	Files	LOC	Purpose
daemon/	8	1,897	APScheduler + jobs
execution/	5	868	Order router + cost model
ingest/	31	4,910	Broker adapters + intel feeds
kernel/	2	177	Boot check
ledger/	27	3,216	Append-only event store
registry/	7	1,267	Strategy versioning + promotion
research/	28	4,906	Backtest + DSR/PBO/BH-FDR + mutation
risk/	21	2,912	Single-entry precheck + SEBI gates
operator_ui/	4	2,440	Dashboard
operator/	5	1,233	CLI helpers
shared/	5	682	Timezone + LLM transport
obs/	3	292	Notifier (Gmail SMTP)
strategies/	—	3,223	9 strategy packages

C.2 Recent commits (India migration)

Commit	Title
86a6778	chore(notifier): point hardcoded Gmail address at operator inbox
a59c67c	feat(strategy): NIFTY_GAP_v1 — overnight gap fade hypothesis at research_only
1ec1759	feat(risk): SEBI gates — index circuit breakers, F&O ban list, price bands, margin
29cf962	feat(india): NSE/BSE migration baseline — Zerodha broker, INR cost model, IST calendar

C.3 TL;DR for either audience

This is a careful, defensive, well-architected trading research lab — not a money-making machine. Its value is the framework (the rules, the safety locks, the hash-chained audit trail, the validation pipeline). Whether it makes money depends entirely on whether the strategies you put through the validation pipeline actually have edge in Indian markets — which is an empirical question this codebase doesn't try to answer; it just gives you an honest way to ask.